Houndstooth

A self-hosted investment tracker built around three design pillars:

- **Privacy-first.** Your financial data lives in *your* Supabase project. The daily AI summary only ever sees anonymized aggregates (sector weights, % moves, tax split) — never dollar balances, share counts, or position values.
- **Single-user.** One person, one instance. No multi-tenant mode, no shared backend, nothing sent to anyone else.
- **Free / low-cost.** Runs on free tiers end to end — Vercel Hobby, Supabase free, SnapTrade's free tier, and Google AI Studio.

It aggregates accounts across institutions, separates taxable vs tax-advantaged holdings, charts your investments over time, and generates a privacy-preserving daily AI market summary.

> **On the name:** Stripe and Plaid were taken — so Houndstooth it is, the next pattern in the drawer.

Stack: Next.js (App Router) · Supabase (Postgres + Auth + RLS) · SnapTrade · Gemini · Tailwind v4 + shadcn/ui · Recharts. Deployed on Vercel.

> **Self-hosting:** Houndstooth is single-user by design — you run your **own** instance with your **own** Supabase project and API keys (Gemini, SnapTrade). Nothing is shared with anyone else. Bug reports are welcome; PRs are not accepted — see CONTRIBUTING.md.

What you'll build

Houndstooth gives you a single dashboard to track your entire financial picture:

- **Net worth over time** — charts showing total assets, debts, investable assets, and net worth across all your accounts.
- **Account breakdown** — every brokerage, bank, or manual entry listed with balances, holdings, and tax classification.
- **AI daily briefing** — a concise market summary tailored to your portfolio's sector exposure and individual stock movers, generated each weekday at 1 PM PT.
- **Manual accounts** — track assets SnapTrade can't reach: home value, credit card debt, 529 plans, RSUs, or anything else.

> **Note:** There is no sign-up page. Users are created manually in your Supabase dashboard (see Full setup step 3). Share the email and password with whoever needs access.

External services you'll need

Service	What it does	Free tier?
Supabase	Hosted PostgreSQL database + authentication + row-level security	✅ Yes
Vercel	Hosting / deployment platform (runs your Next.js app)	✅ Hobby tier
SnapTrade	Aggregates brokerage account data (balances, holdings, transactions) — like Plaid but for investments	✅ Personal tier
Google AI Studio	Provides the Gemini API key for the daily AI summary	✅ Free tier

You can try the app in **mock mode** without any of these. See Quick start below.

Requirements

- **Node ≥ 20** and **npm** (see `.nvmrc`).
- A Supabase project (free tier is fine). SnapTrade + Gemini keys are optional — the app runs fully in **mock mode** without them.

Quick start (mock mode — no credentials needed)

Try the full UI with mock data, before wiring up any external services:

```
git clone <your-fork-url> houndstooth && cd houndstooth
npm install
cp .env.example .env.local        # set DATA_PROVIDER=mock in it
npm run dev                        # http://localhost:3000
```

In `.env.local`, set `DATA_PROVIDER=mock`. You can leave the SnapTrade/Gemini values blank in mock mode. (Supabase auth is still required for login — see Full setup.)

Environment variables

Copy `.env.example` → `.env.local` and fill these in. The same variables are set in your Vercel project for deployment.

Variable	Scope	Required	What happens if skipped	Where to get it
`NEXT_PUBLIC_SUPABASE_URL`	public	yes	App can't connect to the database	Supabase → Project Settings → API
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	public	yes	App can't authenticate or read/write data	Supabase → Project Settings → API
`SUPABASE_SERVICE_ROLE_KEY`	server-only	yes	Cron jobs and server actions fail (bypasses RLS)	Supabase → Project Settings → API (**never expose to the brows**
`DATA_PROVIDER`	server	yes	Must be `mock` or `snaptrade`	—
`SNAPTRADE_CLIENT_ID`	server-only	live only	App runs in mock mode (fake data)	SnapTrade Dashboard → API Keys
`SNAPTRADE_CONSUMER_KEY`	server-only	live only	Same as above	SnapTrade Dashboard → API Keys
`SNAPTRADE_USER_ID`	server-only	live only	Same as above	SnapTrade Dashboard → Settings → Security
`SNAPTRADE_USER_SECRET`	server-only	live only	Same as above	SnapTrade Dashboard → Settings → Security
`GEMINI_API_KEY`	server-only	AI summary	Daily AI summary won't generate (app still works fine)	Google AI Studio (enable Search grounding for the key)
`CRON_SECRET`	server-only	deploy only	Cron jobs won't run on Vercel; skip for local dev	Any random string
`MFA_TRUST_SECRET`	server-only	recommended	MFA trust feature disabled (app still works)	`node -e "console.log(require('crypto').randomBytes(32).toSt`
`NEXT_PUBLIC_ENABLE_ANALYTICS`	public	optional	Vercel Web Analytics stays off (privacy-first default)	Set to `true` to enable

Full setup (live data)

1. Create a Supabase project

1. Go to supabase.com and sign up (free).
2. Click **New Project** → pick a name, set a strong database password, and choose a region closest to you.
3. Wait ~2 minutes for provisioning to complete.
4. Go to **Project Settings** → **API** and copy these three values:
    - `URL` → paste as `NEXT_PUBLIC_SUPABASE_URL`
    - `anon public` key → paste as `NEXT_PUBLIC_SUPABASE_ANON_KEY`
    - `service_role` key → paste as `SUPABASE_SERVICE_ROLE_KEY` (**keep this secret**)

2. Apply the database schema

Run the migrations in order (ascending number). You can do this two ways:

Option A — Supabase SQL Editor (easiest):

1. Go to **SQL Editor** in your Supabase dashboard.
2. Open `supabase/migrations/0001_initial_schema.sql` → copy its contents → paste into the editor → **Run**.
3. Repeat for each migration file: `0002_security_prices.sql`, `0005_daily_summary_single_row.sql`, `0006_account_balances.sql`, `0007_manual_accounts.sql`. (Files 0003 and 0004 are small patches already covered by the full schema.)

Option B — Supabase CLI:

```
npx supabase login
npx supabase link --project-ref <your-project-ref>
npx supabase db push
```

3. Create your user

There is **no sign-up page** in Houndstooth. You create users manually:

1. Go to **Authentication** → **Users** in your Supabase dashboard.
2. Click **Add user** → enter an email and password.
3. The app will automatically create a profile row for them.

> **Tip:** If you're the only user, use your personal email. You'll log in at `/login` with this email + password.

4. Two-factor authentication (MFA) — optional but recommended

Houndstooth uses Supabase's built-in TOTP MFA. Setup happens in the Supabase dashboard:

1. Go to **Authentication** → **Settings** → **MFA** in your Supabase project.
2. Enable **TOTP** (Time-based One-Time Password).
3. For each user, go to **Users** → click the user → **Set up TOTP** → scan the QR code with an authenticator app (Google Authenticator, Authy, etc.).

When MFA is enabled, logging in redirects to a 6-digit code prompt at `/mfa`. Users can check **"Trust this device for 30 days"** to skip MFA on that browser. This requires `MFA_TRUST_SECRET` in your env — generate one with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64url'))"
```

Without it, users must enter a code every login (still secure, just more friction).

5. Set SnapTrade credentials

SnapTrade's free **personal tier** gives you one pre-created user — you don't need to register users in code. Just paste the values from your dashboard:

1. Go to SnapTrade Dashboard.
2. **API Keys** → copy `Client ID` and `Consumer Key` → set `SNAPTRADE_CLIENT_ID` and `SNAPTRADE_CONSUMER_KEY`.
3. **Settings → Security** → copy the `User ID` and `User Secret` for your personal user → set `SNAPTRADE_USER_ID` and `SNAPTRADE_USER_SECRET`.
4. Set `DATA_PROVIDER=snaptrade` in `.env.local`.

6. Set Gemini API key (optional)

For the daily AI summary:

1. Go to Google AI Studio.
2. Sign in with a Google account → go to **API keys** → create a new key.
3. Paste it as `GEMINI_API_KEY` in `.env.local`.
4. Enable **Google Search grounding** for the key (toggle in the API key settings) so the AI can look up real-time market data.

7. Run it

```
npm run dev      # http://localhost:3000  → redirects to /login
npm test         # unit tests
npm run build    # production build
```

Log in with the user you created in step 3. You should see the dashboard with mock data (if `DATA_PROVIDER=mock`) or live brokerage data (if SnapTrade is configured).

Deploy to Vercel

1. **Push your code** to a GitHub repository (fork this repo or clone and push).
2. Go to vercel.com → sign up → **New Project** → **Import** your GitHub repo.
3. In the **Environment Variables** section, add all the variables from the table above. Make sure to include:
   - `CRON_SECRET` — for the daily cron job
   - `MFA_TRUST_SECRET` — recommended for MFA trust
4. Click **Deploy**.
5. After deployment, Vercel gives you a URL like `https://your-app.vercel.app`.

Setting up webhooks (required for automatic sync)

SnapTrade needs to call your app when a brokerage is connected or updated. This only works on a deployed URL — SnapTrade can't reach `localhost`.

1. In Vercel, go to **Settings** → **Domains** and note your deployment URL (or set up a custom domain).
2. In the SnapTrade Dashboard → **Webhooks**, add a new webhook pointing at:

   ```
   https://<your-deployment-url>/api/snaptrade/webhook
   ```

3. Save. SnapTrade will now notify your app automatically when data changes.

Setting up the daily cron

The cron is pre-configured in vercel.json — two weekday runs at 20:10 and 21:10 UTC (covering 1 PM Pacific year-round, including DST). Vercel Hobby allows exactly 2 daily cron jobs.

No additional setup needed beyond setting `CRON_SECRET` in step 3 above.

How it works

Daily snapshot & AI summary

Once per weekday at **1:10 PM Pacific**, the app:

1. Syncs all connected brokerages from SnapTrade (balances, holdings, prices).
2. Refreshes prices for any manually added holdings via Yahoo Finance.
3. Computes today's net-worth snapshot and stores it.
4. Generates an AI market briefing using Gemini 2.5 Flash with Google Search grounding.

The AI summary is **privacy-preserving**: the model only sees sector weights, per-ticker % moves, and the taxable/tax-advantaged split — never dollar balances, share counts, or position values. The output is strictly descriptive (never buy/sell/hold advice).

Webhooks explained

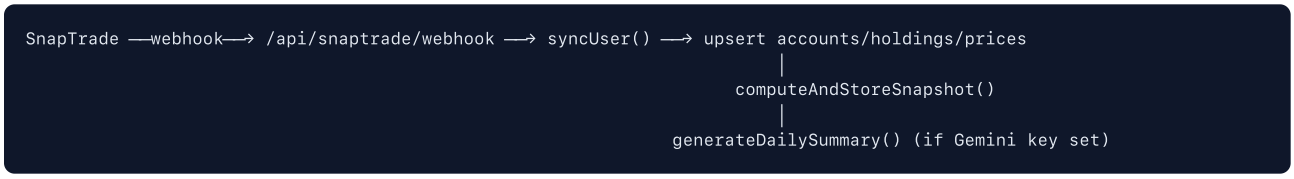
A **webhook** is a URL that SnapTrade calls automatically when something changes (e.g., you connect a new brokerage). It tells your app "hey, new data is ready" so it can pull the latest balances and holdings. Without webhooks, you'd need to manually click "Sync" every time.

On localhost, webhooks won't work (SnapTrade can't reach your machine) — use the **Sync** button in the UI instead. On a deployed Vercel app, webhooks handle this automatically.

SnapTrade connection flow

1. Click **Connect account** in the app → you're redirected to SnapTrade's hosted login page.
2. Sign in to your brokerage through SnapTrade's portal.
3. SnapTrade sends a webhook to your app → accounts and holdings are pulled in automatically.
4. (Optional) You can also connect brokerages directly in the SnapTrade dashboard.

Data flow summary



Local vs deployed

Feature	Local (`npm run dev`)	Deployed (Vercel)
Auth / login	✅	✅
Manual Sync button	✅	✅
SnapTrade webhooks	❌ (use Sync button)	✅
Daily cron	❌ (use `?force=1` query param)	✅
AI summary	✅ (with API key)	✅
Custom domain for webhooks	N/A	✅ (or use Vercel's `.vercel.app` URL)

Troubleshooting / FAQ

Q: I can't find a sign-up page. A: There isn't one. Users are created manually in Supabase → Authentication → Users → "Add user" (see Full setup step 3). Share the email and password with whoever needs access.

Q: My cron job isn't running. A: Make sure `CRON_SECRET` is set in your Vercel project environment variables. Check that the cron jobs appear under Vercel → Settings → Cron. If they show as "disabled", enable them.

Q: The Sync button doesn't pull data. A: Verify that `DATA_PROVIDER=snaptrade` and all four SnapTrade keys (`CLIENT_ID` , `CONSUMER_KEY` , `USER_ID` , `USER_SECRET`) are correct. Check the Vercel deployment logs (or `npm run dev` output) for error messages.

Q: I connected a brokerage but it doesn't show up. A: If deployed, check that your SnapTrade webhook URL is correct and reachable. If local, use the **Sync** button in the UI — webhooks can't reach localhost.

Q: The AI summary says "skipped_no_key". A: Make sure `GEMINI_API_KEY` is set and that Google Search grounding is enabled for that key in Google AI Studio.

Q: I see mock/fake data everywhere. A: Check that `DATA_PROVIDER=snaptrade` (not `mock`) in your `.env.local` or Vercel env settings, and that all SnapTrade keys are populated.

Q: Webhooks return 401 or 500 errors in the SnapTrade dashboard. A: The webhook route requires a valid HMAC signature. Make sure `SNAPTRADE_CONSUMER_KEY` is set correctly — it's used to verify incoming signatures. Also check your deployment logs for the full error.

Q: Can I run this with multiple users? A: No — Houndstooth is single-user by design. Each instance serves one person with one Supabase project.

Foundation

- Next.js + Tailwind + shadcn scaffold with a dark-first theme.
- Supabase clients: browser (RLS), server (RLS), admin (service-role, server-only).
- Single-user email auth + `proxy.ts` route protection.
- Full schema migration with RLS on every table (`supabase/migrations/0001_initial_schema.sql`).
- Dashboard shell with **two hero charts** (Net Worth + Investments), shared range pills, and allocation cards.

Conventions

- Never expose the SnapTrade `userSecret` to the browser — it stays server-only (env), used exclusively by Route Handlers / cron via the service-role client.
- Net worth is computed server-side and snapshotted; the frontend only reads derived data.